

Why `BaseModel` and `Field` Matter

A LangChain `create_agent` example, with Pydantic doing the real work

The core idea in one sentence: **Pydantic is how the LLM learns what your tool expects.** `BaseModel` defines the shape, `Field` supplies the descriptions and constraints, and both are compiled into the JSON schema the model actually sees at tool-call time.

Setup

```
pip install langchain langchain-tavily pydantic
export TAVILY_API_KEY="tvly-..."
export OPENAI_API_KEY="sk-..."
```

The example

One file, two Pydantic models — one governing what goes *into* the tool, one governing what comes *out of* the agent.

```
from typing import Literal
from pydantic import BaseModel, Field, field_validator
from langchain_core.tools import tool
from langchain.agents import create_agent
from langchain_tavily import TavilySearch

# — 1. BaseModel + Field define the TOOL'S INPUT contract —————
class WeatherQuery(BaseModel):
    """Arguments for looking up weather in a city."""

    city: str = Field(
        description="City name only, no country or state. "
        "E.g. 'Hyderabad', not 'Hyderabad, India'."
    )
    country_code: str = Field(
        description="ISO 3166-1 alpha-2 country code, e.g. 'IN', 'US', 'GB'.",
        min_length=2,
        max_length=2,
    )
    units: Literal["metric", "imperial"] = Field(
        default="metric",
        description="'metric' for Celsius, 'imperial' for Fahrenheit.",
    )
    forecast_days: int = Field(
        default=1,
        ge=1,
        le=7,
        description="How many days ahead to forecast. 1 means today only.",
    )

    @field_validator("country_code")
    @classmethod
    def upper(cls, v: str) -> str:
        return v.upper()
```

```

search = TavilySearch(max_results=3, include_answer=True)

@tool(args_schema=WeatherQuery)
def lookup_weather(city: str, country_code: str, units: str = "metric",
                   forecast_days: int = 1) -> str:
    """Look up current weather and short-term forecast for a city."""
    unit_word = "celsius" if units == "metric" else "fahrenheit"
    query = f"weather {city} {country_code} next {forecast_days} days in {unit_word}"
    return search.invoke({"query": query})["answer"]

# — 2. BaseModel + Field define the AGENT'S OUTPUT contract —
class WeatherReport(BaseModel):
    city: str = Field(description="The city the report is for.")
    temperature: float = Field(description="Current temperature, in the requested units.")
    units: Literal["C", "F"]
    conditions: str = Field(description="Short phrase: 'light rain', 'clear', 'overcast'.")
    humidity_pct: int | None = Field(default=None, ge=0, le=100)
    advice: str = Field(description="One practical sentence, e.g. 'Carry an umbrella.'")

agent = create_agent(
    model="openai:gpt-4o-mini",
    tools=[lookup_weather],
    system_prompt="You are a weather assistant. Use the tool, then report clearly.",
    response_format=WeatherReport,
)

result = agent.invoke({
    "messages": [{"role": "user", "content": "Do I need a jacket in Hyderabad today?"}]
})

report: WeatherReport = result["structured_response"]
print(f"{report.city}: {report.temperature}°{report.units}, {report.conditions}")
print(report.advice)
print(report.model_dump_json(indent=2))  # ready for an API response or DB write

```

Why it matters — see what the model receives

The models above are not decoration. Ask Pydantic to show you the schema it generates:

```
print(WeatherQuery.model_json_schema())
```

```

{
  "properties": {
    "city": {
      "description": "City name only, no country or state. E.g. 'Hyderabad', not 'Hyderabad, India'.",
      "type": "string"
    },
    "country_code": {
      "description": "ISO 3166-1 alpha-2 country code, e.g. 'IN', 'US', 'GB'.",
      "maxLength": 2,
      "minLength": 2,
      "type": "string"
    },
    "units": {
      "default": "metric",
      "enum": ["metric", "imperial"],
      "type": "string"
    }
  }
}

```

```

    },
    "forecast_days": {
        "default": 1,
        "maximum": 7,
        "minimum": 1,
        "type": "integer"
    }
},
"required": ["city", "country_code"]
}

```

Those `description` strings are **prompt text**. Without them the model guesses — it will pass `"Hyderabad, Telangana, India"` as `city`, or `"India"` as `country_code`, and your tool breaks. With them, it fills the arguments correctly on the first attempt.

The four jobs Pydantic is doing

CONSTRUCT	WHAT IT DOES	WHAT BREAKS WITHOUT IT
<code>BaseModel</code>	Generates the JSON schema sent to the model for tool calling	The LLM has no idea what arguments exist or what types they take
<code>Field(description=...)</code>	Documents each argument to the LLM — this text <i>is</i> prompt text	Wrong-format arguments, retry loops, wasted tokens
<code>Field(ge=, le=, min_length=)</code>	Validates values before your function body ever runs	<code>forecast_days=90</code> reaches your API and 500s
<code>response_format=Model</code>	Returns a typed object instead of a string	You regex the model's prose and hope the format holds

The contrast, in one line

```

@tool
def lookup_weather_bad(city: str, cc: str, u: str = "metric", d: int = 1) -> str:
    """Get weather."""

```

Same function, no Pydantic model. The LLM sees `cc`, `u`, `d` with no explanations and a one-word docstring — so it invents values for all three. That is the entire argument for `BaseModel` and `Field`.

Takeaways

- **Write descriptions as if for a new teammate.** Every `Field` description is instruction the model reads at call time; vague ones cost you retries.
- **Put constraints in the schema, not in your function body.** `ge`, `le`, `min_length` and `Literal` both guide the model and reject bad values before your code runs.
- **Use `response_format` whenever output feeds other code.**
`result["structured_response"].temperature` is a real `float` you can compare against a threshold

— no parsing, and the guarantee is enforced, not hoped for.

- **Literal beats str for enums.** It becomes an `enum` in the schema, which removes an entire class of invalid input.